

# Evaluating your sanitiser with "evil-origin"

## *Project testing guide*

### 1. Project testing and verification

The web-sanitiser final project needs to be solidly tested and verified in order to make sure that its implementation is correct.

In order to do this, you will need to have access, in some way, to a source that serves malicious web content that matches the scenarios outlined in the project reference. A possible way to achieve this is to host a public site, reachable by anyone, that responds with the aforementioned content; however, this is both dangerous and ill-advised, mostly because it could be exploited by bad faith actors and misused by students and other people alike.

To avoid this issue, it was decided to give access to this testing and verification tool in a different way, containerizing it and distributing it as a multi-platform Docker image.

**evil-origin** is the specialized web server designed for this purpose in this project. It hosts a catalog of "scenarios"-files and responses specifically crafted to trigger security vulnerabilities like XSS, SSRF, and Zip Bombs. By pointing your sanitiser at this server, you can measure how well your code detects and neutralizes these threats in a safe, controlled environment.

The **evil-origin** web-server is purely a testing tool and as such it must only be used to verify your sanitiser's correctness and nothing else. You must not use it for unintended and/or malicious purposes.

### 2. Why Docker?

Docker is a platform to develop, ship, and run containerized applications. It is a powerful tool that allows us to easily distribute programs or apps without having to worry about setting up the entire environment (like specific Node.js versions or libraries) on the host machine.

This approach allows mostly complete execution isolation, reproducibility, and portability. If we provided the raw source code, you might encounter "it works on my machine" errors. By using Docker, we guarantee that the **evil-origin** server behaves exactly the same way for every student, regardless of their operating system. *(If you want to extend your knowledge about containers or images, you can read more about them on the official Docker website [here](#) and [here](#)).*

### 3. Pre-requisites

Along with this document, you will find a compressed **.zip** file (called evil-origin.zip) which contains the **.tar** archive of a Docker image of a web application that will be used to test and

verify your sanitiser's correctness and featurefulness (called `evil-origin.tar`). To be able to run the application contained within the Docker image, you will need to follow a few steps, after having made sure to have the necessary pre-requisites.

Your machine must meet one of the following hardware and OS combinations:

- An **x64 CPU**, running either Windows 10/11, macOS, or a Linux distribution among Ubuntu, Debian, Red Hat, or Fedora.
- An **ARM CPU**, running either Windows 11 or macOS.

*(You can find a more detailed compatibility list for Docker on their website).*

Additionally, you will need to install the following programs:

- WSL (Windows only) - [install guide](#)
- Docker Desktop - [install guide](#)

## 4. Getting started (operational details)

### 4.0. Extract the .tar archive from the zip

First, you'll need to decompress the zip file that contains the tar archive. Doing so will create a folder called `evil-origin`, with a file inside called `evil-origin.tar`. This archive file is the Docker image which you'll work with. You **do not** need to extract the tar archive, Docker will load it as-is.

### 4.1. Loading the image

Then, you need to load the provided project environment into your Docker library. To do so, from a terminal opened *in the same folder* as the tar archive, run the following command:

```
Shell
docker load -i evil-origin.tar
```

### 4.2. Starting the Project Server

Once loaded, you must run the container. This starts the testing environment on your machine.

#### Option A - Docker Desktop (UI)

1. Open Docker Desktop and go to the **Images** tab.
2. Find `evil-origin` and click **Run**.
3. In "Optional Settings" ensure the Local Port is set to `3100`.

#### Option B - Terminal (command line)

1. Run the following command to start the server in the background:

Shell

```
docker run -d -p 3100:3100 --name evil-origin evil-origin
```

### 4.3. Verifying the Connection

To ensure the project server is ready, send a "health check" request:

None

```
curl http://localhost:3100/health
```

If you see the following message:

JSON

```
{"status": "ok", "service": "evil-origin"}
```

your environment is now active and ready.

## 5. The testing workflow

### 5.1. Discovering threats

The server contains a catalog of attack scenarios. You can view the list of available threats by requesting the `/scenarios` endpoint.

To format the output cleanly, we recommend using a tool called `jq`.

- **Windows (via winget):** `winget install jqlang.jq`
- **macOS (via Homebrew):** `brew install jq`
- **Linux (apt):** `sudo apt install jq`
- **Via the [official website](#)**

List the scenarios with the command:

None

```
curl -s http://localhost:3100/scenarios | jq
```

*(if you cannot install jq, simply run the command without it to see the raw text).*

### 5.2. Performing an evaluation

To test your sanitiser, you shouldn't download the files manually. Instead, tell your sanitiser to fetch the malicious URL from the **evil-origin** server.

#### Example: testing a basic XSS script

None

```
curl -s -X POST http://localhost:3000/v1/resources \
-H "Content-Type: application/json" \
-d '{"url": "http://localhost:3100/html/script-tag"}'
```

In this flow, assuming your sanitiser is working correctly:

1. Your **sanitiser** (on port 3000) receives the request.
2. It fetches the hostile content from **evil-origin** (on port 3100).
3. It applies your *Rust* logic to clean the content.
4. It returns a report to you.

## 6. Catalog of attack classes

Below are the primary categories of threats you are expected to defend against, extracted from a subset obtained by hitting the **/scenarios** endpoint of the **evil-origin** server

- **XSS & Injection:** Attempts to execute unauthorized JavaScript in the browser.
  - *Key Scenarios:* **/html/script-tag**, **/html/inline-handler**
- **MIME Confusion:** Files that lie about their type (e.g., a script disguised as an image).
  - *Key Scenarios:* **/mime/html-disguised-as-png**
- **SSRF:** Attempts to trick your sanitiser into attacking your internal network.
  - *Key Scenarios:* **/html/ssrf-internal-reference**
- **Resource Bombs:** Payloads designed to crash your sanitiser (e.g., "Billion Laughs" or huge images).
  - *Key Scenarios:* **/mime/xml-bomb**, **/image/huge-dimensions**
- **Credential Leakage:** Verifies that your sanitiser doesn't accidentally forward sensitive cookies to the "evil" server.
  - *Key Scenarios:* **/html/echo-headers**

(For a full list of the 28 available scenarios, use the **/scenarios** endpoint described in section 5.1).